

Flask Online Shop — Deep Dive

Explainer document generated from the project source

Contents

1	What this project is	1
2	Directory layout	3
3	High-level architecture	4
4	The product catalog and price/display consistency	4
4.1	Two dictionaries, one source of truth	4
4.2	Why this matters: a real bug this design fixes	5
5	Data model and database	5
5.1	The <code>User</code> table	5
5.2	Where the database file lives	6
6	Authentication flows	7
6.1	Registration (<code>/add_user</code>)	7
6.2	Login (<code>/login</code>) and the bug that was fixed here	7
6.3	What “logged in” actually means here	8
6.4	Logout (<code>/logout</code>)	8
7	The flashed-message pattern	9
8	The checkout (payment) flow	9
9	Templates and the design system	10
9.1	Template inheritance	10
9.2	Design system in <code>styles.css</code>	11
10	Configuration and secrets	11
11	Dependencies and their roles	12
12	Flagged ambiguities and honest gaps in this document	12

1 What this project is

Flask Online Shop is a small e-commerce demonstration website. A visitor can browse a catalog of nine “planets” (used as stand-in products), create an account, log in, and click “Buy” on a

planet to be sent to a hosted Stripe Checkout page to pay for it. The whole application is a single Python file, `main.py`, plus a handful of HTML templates and one CSS file. There is no separate backend/frontend split, no build step, and no JavaScript framework: it is a classic “server renders HTML, browser submits forms” web application.

Key beginner terms used throughout this document

Web framework A library that handles the repetitive parts of building a website for you: turning an incoming HTTP request into a call to your Python function, and turning your function’s return value into an HTTP response. This project uses **Flask**, a small, unopinionated Python web framework.

Route A rule that maps a URL path (like `/login`) and an HTTP method (GET or POST) to a Python function that handles it.

HTTP method The verb of an HTTP request. GET means “give me this page”; POST means “here is data, please act on it” (e.g. submitting a form).

Template / templating engine An HTML file with placeholders that get filled in with real data at render time. Flask uses **Jinja2**; placeholders look like `{{ variable }}` and control structures look like `{% if ... %}`.

ORM (Object-Relational Mapper) A library that lets you work with database rows as if they were ordinary Python objects, instead of writing raw SQL by hand. This project uses **Flask-SQLAlchemy**.

SQLite A database engine that stores an entire database as a single file on disk, with no separate server process to run. Good for small projects and demos.

Session / cookie HTTP is stateless: each request is independent and the server would not normally remember who you are between requests. A **cookie** is a small piece of data the browser stores and sends back on every request to the same site. Flask’s **session** object is a dictionary-like store whose contents are saved into a signed cookie, letting the server “remember” things (like a flash message) across requests from the same browser.

Hashing (passwords) A one-way mathematical transformation. Instead of storing a user’s actual password, the server stores the output of a hash function applied to it. Later, to check a login attempt, the server hashes the submitted password the same way and compares the two hashes — the real password is never stored or compared directly. This project uses **bcrypt**, a hashing algorithm designed specifically for passwords (deliberately slow, to resist brute-force guessing).

CSRF (Cross-Site Request Forgery) An attack where a malicious website tricks a logged-in user’s browser into submitting a request to a different site (e.g. your bank) without the user’s intent. Frameworks usually defend against this with a hidden, per-session token in every form. This project’s forms do *not* include a CSRF token (see Section 10).

Environment variable A named value passed to a program from outside its source code (from the shell, a `.env` file, or the hosting platform), used here to keep secrets (like API keys) out of the codebase.

API key / secret key A private string that authenticates a program to an external service. Stripe’s secret key authenticates this app’s server to Stripe’s API; Flask’s secret key

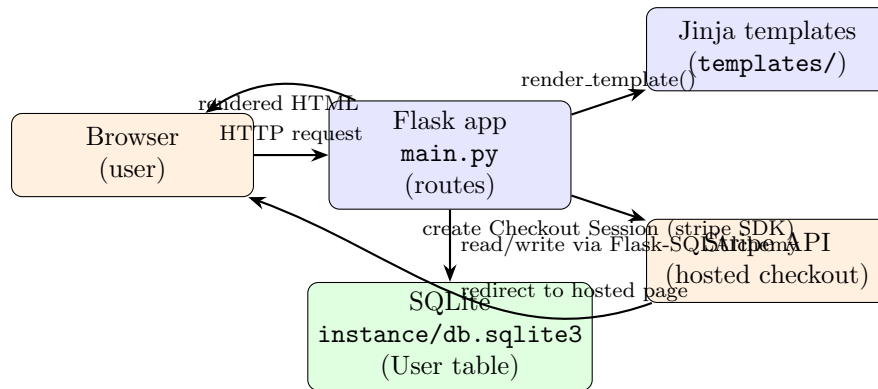
is used to cryptographically sign session cookies so a client cannot forge or tamper with them.

2 Directory layout

```
flask-online-shop/
├── main.py (all routes, models, business logic)
├── requirements.txt (pinned Python dependencies)
├── .env.example (documents required env vars)
├── README.md
├── LICENSE
├── templates/
│   ├── base.html (shared layout: nav, flash messages, footer)
│   ├── index.html (catalog / home page)
│   ├── login.html
│   ├── create_user.html (registration page)
│   ├── success.html
│   └── cancel.html
├── static/
│   ├── css/styles.css (design system)
│   └── images/*.jpeg (9 planet photos)
└── instance/ (created at runtime; holds db.sqlite3, not in git)
```

Flask, by default, looks for a `templates/` folder and a `static/` folder next to the file that creates the Flask app object. This project makes that explicit by computing `base_path` from `main.py`'s own file location (`os.path.dirname(os.path.abspath(__file__))`) and passing `template_folder/static_folder` in explicitly, so the app finds its templates and static files regardless of the working directory the process is started from.

3 High-level architecture



The application is organized around four responsibilities, all living in `main.py`:

1. **Product/display data:** two dictionaries, `planet_price_dict` (the pricing data shape Stripe requires) and `PLANET_INFO` (name, image, description), combined by `get_planets()` into the list the catalog template loops over.
2. **Persistence:** a single SQLAlchemy model, `User`, backed by a SQLite file created automatically on startup.
3. **Auth:** registration (`/add_user`) and login/logout (`/login`, `/logout`) routes, using `bcrypt` for password hashing.
4. **Payment:** one route, `/create-checkout-session`, that asks Stripe to create a Checkout Session for a single planet and redirects the browser to Stripe’s own hosted payment page.

4 The product catalog and price/display consistency

4.1 Two dictionaries, one source of truth

`planet_price_dict` is shaped exactly the way Stripe’s `checkout.Session.create` API expects a line item: a `price_data` object with `currency`, a nested `product_data.name`, and `unit_amount` (the price in the smallest currency unit — cents for USD), plus a `quantity`. `PLANET_INFO` holds only display copy: the human-readable name, the image filename, and a description. Neither dictionary alone is enough to render a product card with a correct price, so `get_planets()` joins them:

```
1 def get_planets():
2     planets = []
3     for slug in PLANET_ORDER:
4         info = PLANET_INFO[slug]
5         price_cents = planet_price_dict[slug]['price_data']['unit_amount']
6         planets.append({
7             'slug': slug,
8             'name': info['name'],
9             'image': info['image'],
10            'description': info['description'],
11            'price_display': format_price(price_cents),
12        })
13     return planets
```

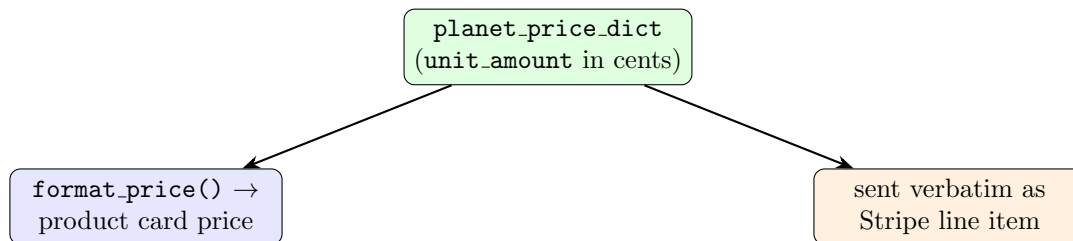
PLANET_ORDER is a plain list of slugs that fixes the display order (Mercury through Pluto), since Python dict iteration order, while stable since Python 3.7, is insertion order rather than an explicitly declared display order.

4.2 Why this matters: a real bug this design fixes

Before this refactor, the home page template hardcoded a price string per planet (e.g. "\$69,420" for Uranus) that was typed independently of `planet_price_dict`. Uranus's actual `unit_amount` in the dictionary that gets sent to Stripe is 4206900 cents = \$42,069 — a different number from what the page displayed. Every other planet's hand-typed price happened to match its Stripe amount by coincidence, so the mismatch was invisible without checking each one individually. `format_price()` removes the possibility of this class of bug entirely, because there is only one number (the one in `planet_price_dict`) and the display string is always derived from it:

```
1 def format_price(unit_amount_cents):
2     dollars = unit_amount_cents / 100
3     if dollars == int(dollars):
4         return f'${int(dollars):,}'
5     return f'${dollars:,.2f}'
```

This is why Earth displays as \$777,777.77 (from `unit_amount` = 77777777) with cents shown, while Mercury displays as \$10,000 (from `unit_amount` = 1000000) with no cents — the function only shows cents when they are non-zero.



5 Data model and database

5.1 The User table

```
1 class User(db.Model):
2     id = db.Column(db.Integer, primary_key=True)
3     email = db.Column(db.String(100))
4     password = db.Column(db.String(100))
5     logged_in = db.Column(db.Integer)
```

Column	Type	Meaning
id	integer, primary key	Auto-incrementing row identifier, assigned by SQLAlchemy/SQLite.

<code>email</code>	<code>string(100)</code>	The user's email address, used as the login identifier. There is no uniqueness constraint declared on this column at the database level, so in principle two rows could share an email (not observed as an issue in the flows tested, but worth flagging — see Section 10).
<code>password</code>	<code>string(100)</code>	The <i>bcrypt hash</i> of the user's password, never the plaintext password itself.
<code>logged_in</code>	<code>integer</code>	1 if this user is currently considered logged in, 0 (or unset) otherwise. This is a property of the <i>database row</i> , not of a browser session — see Section 6.3 for what that implies.

5.2 Where the database file lives

```

1 base_path = os.path.dirname(os.path.abspath(__file__))
2 instance_dir = os.path.join(base_path, 'instance')
3 os.makedirs(instance_dir, exist_ok=True)
4 path_to_sql_file = os.path.join(instance_dir, 'db.sqlite3')
5 app.config['SQLALCHEMY_DATABASE_URI'] = f'sqlite:/// {path_to_sql_file}'

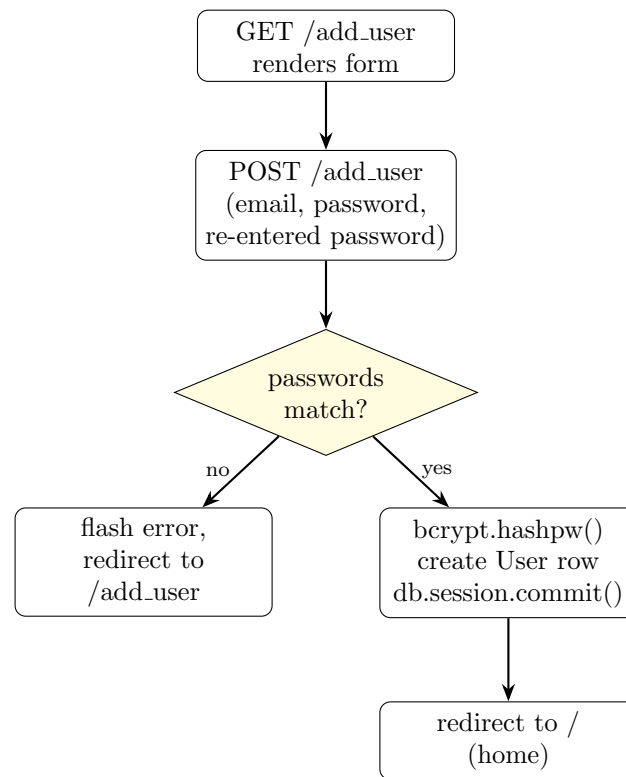
```

`__file__` is a Python built-in variable holding the path of the currently executing file, so `base_path` always resolves to the folder containing `main.py` itself, no matter what directory the process was launched from. This replaced an earlier version that derived the same path from `sys.argv[0]` (the command used to launch the script), which breaks if the script is invoked a different way (e.g. through a process manager or a different working directory) — see the **Challenges** section of the project README for the full before/after.

`db.create_all()` is called once, inside an `app.app_context()` block, at import time. It creates any tables declared as `db.Model` subclasses that do not already exist in the SQLite file; it does nothing (no error, no data loss) if they already exist, which is why it is safe to run unconditionally on every process start rather than checking first.

6 Authentication flows

6.1 Registration (/add_user)



Key points:

- The password is converted to bytes (`bytes(pw, 'UTF-8')`) because `bcrypt`'s functions operate on byte strings, not Python's native `str` type.
- `bcrypt.gensalt()` generates a random *salt* — extra random data mixed into the hash so that two users with the same password get different stored hashes, and so precomputed “rainbow table” attacks do not work.
- The new user is *not* automatically logged in after registration; they are sent to the login page implicitly by being redirected home, then must log in separately.
- The registration form's password field has `maxlength="10"` in the HTML, but this is a client-side-only constraint (see Section 10).

6.2 Login (/login) and the bug that was fixed here

```
1 user = User.query.filter_by(email=email).first()
2 if user:
3     encoded_pw = bytes(pw, 'UTF-8')
4     check = bcrypt.checkpw(encoded_pw, user.password.encode('UTF-8')
5                     if isinstance(user.password, str) else user.password)
6     if check:
7         flash('You are now Logged In')
8         user.logged_in = 1
9         db.session.commit()
```

```

10     return redirect(url_for('home'))
11     else:
12         error = 'That password is incorrect'
13         return render_template('login.html', error=error)
14 else:
15     error = 'That email is not in our system'
16     return render_template('login.html', error=error)

```

`bcrypt.checkpw(plaintext_bytes, stored_hash_bytes)` recomputes the hash of the submitted password using the salt embedded in the stored hash, and compares the result to the stored hash, returning `True/False`. The row looked up is the *specific row matching the submitted email* (`User.query.filter_by(email= email).first()`) — this is the fixed version. The README documents that the original code looked up whether *any* user existed with that email, but then always re-fetched and compared against *user id 1*’s password hash regardless of which email was submitted. With exactly one row in the table this bug is invisible (user 1 *is* the only user, so “the submitted email’s row” and “row 1” are the same row); the moment a second account exists, logging in as user 2 would be checked against user 1’s hash — letting a wrong password through if it happened to match user 1’s, or rejecting user 2’s own correct password. This is a good illustration of a bug class: code that hardcodes an identity (“row 1”) instead of deriving it from the actual request (“the row for this email”) often passes casual, single-user testing and fails only once a second real actor is involved.

6.3 What “logged in” actually means here

```

1 def is_logged_in():
2     return bool(db.session.query(exists().where(User.logged_in == 1)).scalar())
3
4 @app.context_processor
5 def inject_nav_state():
6     return {'logged_in': is_logged_in()}

```

A **Flask context processor** is a function registered with `@app.context_processor` that Flask calls automatically before rendering *any* template, injecting whatever dictionary it returns into that template’s variables. Here it injects `logged_in` into every template’s context, which is why `base.html`’s nav (shared across all pages) can show “Login” or “Logout” correctly no matter which route rendered the current page.

The important nuance: `is_logged_in()` asks “*does any row in the User table have logged_in = 1?*” — it is a global, table-wide flag, not tied to the browser making the request. This means, architecturally, that “being logged in” is a property of a *user account row*, not of a *browser session*. In a single-browser demo this is invisible, but it means two different browsers could observe each other’s login state, and it is explicitly called out in the README’s “What I’d do differently” as something a real `flask-login`-based session system would fix.

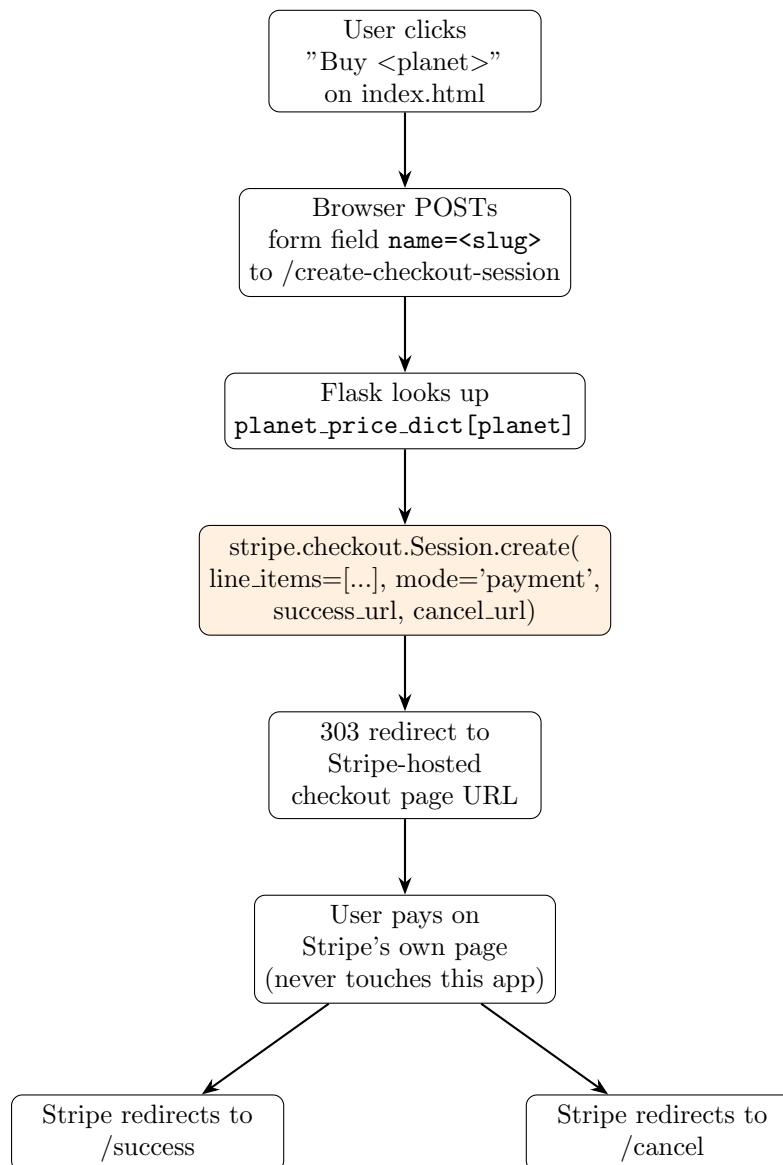
6.4 Logout (/logout)

Finds whichever user row currently has `logged_in = 1`, flips it back to 0, commits, flashes a message, and redirects home. Note this route accepts GET (a plain link, as used in the nav), which is a minor deviation from the common best practice of using POST for state-changing actions, but is consistent with this being a demo rather than a hardened production app.

7 The flashed-message pattern

`flash(message)` stores a string in the session’s message queue (backed by the signed cookie). `get_flashed_messages()`, called once inside `base.html`, retrieves and *clears* that queue on the very next render, so a flashed message is shown exactly once, on the page the user lands on right after the action that triggered it (e.g. “You are now Logged In” appears once on the home page immediately after a successful login, then disappears on subsequent visits). `session.pop('_flashes', None)` at the top of the `/login` GET/POST handler explicitly discards any stale queued flash messages before rendering the login form fresh, so a leftover flash from an unrelated earlier action cannot bleed into the login page.

8 The checkout (payment) flow



```
1 @app.route('/create-checkout-session', methods=['POST'])
2 def create_checkout_session():
```

```

3  planet = request.form.get('name')
4  checkout_session = stripe.checkout.Session.create(
5      line_items=[planet_price_dict[planet]],
6      mode='payment',
7      success_url=f'{YOUR_DOMAIN}/success',
8      cancel_url=f'{YOUR_DOMAIN}/cancel',
9  )
10 return redirect(checkout_session.url, code=303)

```

A **303 See Other** redirect specifically tells the browser to re-request the target URL with **GET** regardless of the original request’s method — the correct choice here since the original request was a **POST** (form submission) but the target (Stripe’s hosted page) should be fetched with **GET**.

There is deliberately **no cart**: each “Buy” button starts a brand-new, single-item Checkout Session scoped to exactly one planet. Buying two planets means two separate trips through this flow.

What was and was not verified for this flow (stated plainly, not embellished)

Per the project README:

- **Verified:** `POST /create-checkout-session` was called directly with a valid planet name and *no* Stripe key configured. The request reached `stripe.checkout.Session.create` and failed only at Stripe’s own authentication check (“no API key provided”), which confirms the route, the line-item lookup, and the call into the Stripe SDK are wired correctly up to the point where a real secret key is required.
- **Not verified:** a full, real Stripe checkout end to end — actually creating a live-against-Stripe test session, completing a test payment on Stripe’s hosted page, and landing on `/success` as a result of a real payment. This was not exercised because this project’s environment never had a real Stripe test secret key available. The checkout route was exercised structurally (as above) and its shape matches Stripe’s documented Checkout Session API, but the full round trip through Stripe’s servers has not been observed to work.
- Because there is no Stripe webhook handler, `/success` is just a page that renders unconditionally when visited — a user could navigate to `/success` directly in their browser without ever paying, and the app would have no way to tell the difference. This is a known, stated limitation (see the README’s “What I’d do differently”), not a hidden one.

9 Templates and the design system

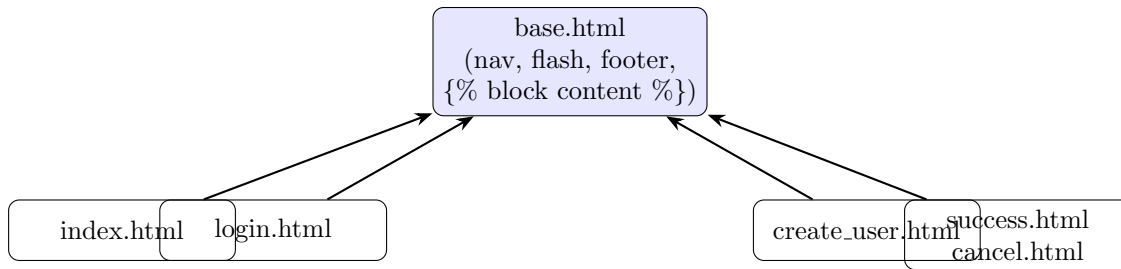
9.1 Template inheritance

```

1  {% extends 'base.html' %}
2  {% block content %}
3      ...page-specific markup...
4  {% endblock %}

```

`base.html` defines named `{% block %}` regions (`head`, `content`, `footer`). Every other template starts with `{% extends 'base.html' %}` and only fills in the blocks it needs to override (usually just `content`, sometimes `title`). This means the nav bar, flashed-message rendering, Bootstrap/Stripe.js script tags, and footer are written once and automatically present on every page, rather than copy-pasted into five files.



9.2 Design system in styles.css

The stylesheet is layered on top of Bootstrap 5 (loaded from a CDN in `base.html`) rather than replacing it, using CSS **custom properties** (variables), declared once under `:root` and referenced everywhere with `var(--name)`:

```

1 :root {
2   --color-bg: #131a2b; /* page background, dark navy */
3   --color-primary: #5b8def; /* buttons, links, focus rings */
4   --color-accent: #f2b84b; /* the "Store" in "PlanetStore" */
5   --color-surface: #f4f6f8; /* light card backgrounds */
6   ...
7 }

```

Reusable component classes built on those variables:

- `.product-card` / `.product-grid`: the catalog's responsive card grid (`grid-template-columns: repeat(auto-fill, minmax(280px, 1fr))` lets the number of columns adapt to viewport width automatically).
- `.auth-card`: shared visual shell for the login and registration forms.
- `.result-card`: shared visual shell for the success and cancel pages, differentiated only by a green checkmark vs. a red cross icon (`.result-icon.success` / `.result-icon.cancel`).
- `.alert-flash`: the box a flashed message renders inside of.

This is why all five page types (catalog, login, register, success, cancel) share one color palette, one card shape (`--radius: 14px`), and one shadow style (`--shadow`), despite being visually distinct pages.

10 Configuration and secrets

```

1 SECRET_KEY = os.environ.get('FLASK_SECRET_KEY', 'dev-only-insecure-secret-key')
2 STRIPE_SECRET_KEY = os.environ.get('STRIPE_SECRET_KEY', '')
3 ...
4 YOUR_DOMAIN = os.environ.get('YOUR_DOMAIN', 'http://127.0.0.1:5000')

```

All three values are read from environment variables, with the `.env.example` file documenting what each one is for and how to generate/obtain it. Per the README's Challenges section, this was a deliberate fix: the original code had both a real-looking hardcoded Stripe test key and a hardcoded Flask signing key checked directly into the source file, which is a well-known class of security mistake (secrets committed to version control can leak permanently, even if removed in a later commit, because they remain in git history unless that history is rewritten).

Known, stated security/robustness gaps (not fixed, and not claimed to be)

- No CSRF token on the login/registration/checkout forms.
- Password length is capped only by an HTML `maxlength` attribute on the registration form — trivially bypassed by anyone not using the browser form (e.g. a direct HTTP request), since the server performs no length check itself.
- No server-side email format validation; the browser’s HTML5 `type="email"` check is the only gate, and it is skipped entirely by a direct request.
- “Logged in” is a global, per-row database flag rather than a per-browser session (Section 6.3), so it does not provide real session isolation between two different browsers or devices.
- `/success` is reachable directly, with no server-side confirmation that a payment actually happened (no Stripe webhook handler).

These are documented here exactly as the project’s own README documents them, not inferred or embellished.

11 Dependencies and their roles

Package	Version	Role
Flask	3.0.3	The web framework: routing, request/response objects, templating integration, session/cookie handling, the flash-message system.
Flask-SQLAlchemy	3.1.1	Binds SQLAlchemy (below) to a Flask app: gives <code>db.Model</code> , <code>db.session</code> , and reads database config from <code>app.config</code> .
SQLAlchemy	2.0.30	The underlying ORM/SQL toolkit that Flask-SQLAlchemy wraps; used directly here for the <code>exists()</code> query helper in <code>is_logged_in()</code> .
stripe	9.12.0	Official Stripe Python SDK; used for exactly one call, <code>stripe.checkout.Session.create(...)</code> .
bcrypt	4.1.3	Password hashing (<code>hashpw/checkpw</code>) used in registration and login.

12 Flagged ambiguities and honest gaps in this document

- No automated test suite exists in this repository; all “verified” claims in this document are carried over verbatim from the project README’s own account of manual verification (a running dev server driven with a `requests` session), not re-verified independently while producing this PDF.
- The full Stripe checkout round trip (real payment, landing on `/success` from an actual completed Stripe session) has never been exercised in this project, for the stated reason that no real Stripe test-mode secret key has been available. This document does not claim otherwise anywhere.
- The `User.email` column has no explicit `unique=True` constraint or index in the model definition; this is inferred directly from reading `main.py` and is not called out explicitly in the

README, so it is flagged here as an observation rather than a documented, intentional design choice.